

HamNoSys Generation Project – Complete Study Guide

(For understanding the report deeply enough to defend it in a viva)

0. The One-Paragraph Version

Sign language has no standard written form. HamNoSys is a symbolic notation (like phonetic spelling, but for signs) that describes **handshape, orientation, location, and movement**. Writing it by hand is slow and inconsistent. This project **automates** that writing process: it takes a photo/video of someone signing, uses **MediaPipe** to find 21 points (landmarks) on the hand (and body/face landmarks too), and runs those points through four independent modules – one per HamNoSys parameter – to predict the correct symbols. It then **checks its own work** by turning the predicted notation into an animated 3D avatar and comparing the avatar's gesture back to the original video using cosine similarity.

Two different techniques are used, on purpose:

- **Handshape** → machine learning (hierarchical classifiers), because the team could build a labeled image dataset.
- **Orientation, Location, Movement** → pure geometry/math on landmark coordinates, because no labeled video dataset for these exists publicly, and rule-based geometry needs zero training data.

Keep that split in your head – it's the single most-asked viva question.

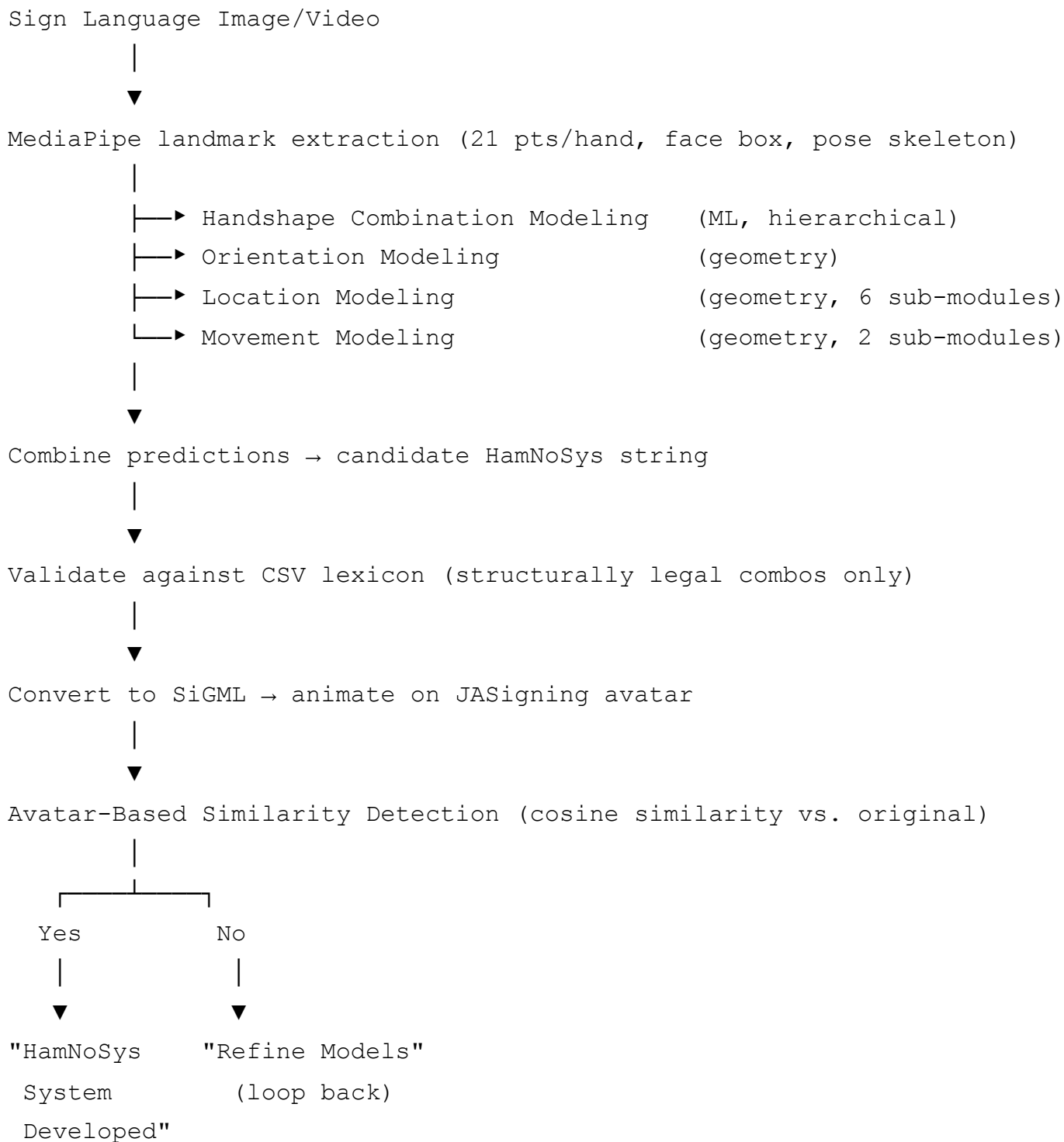
1. Why This Project Exists

- Spoken languages have alphabets; sign languages don't have an equivalent standard writing system.
- HamNoSys (Hamburg Notation System) fills that gap, but transcription is **manual** – trained experts watch video and write symbols frame by frame. Slow, expensive, inconsistent, doesn't scale.
- Existing automated approaches mostly handle **isolated single signs**, not continuous signing (transitions, two-hand overlap, co-articulation).
- Nobody had a reliable way to **verify** that an automatically generated HamNoSys string actually matches the original gesture.

The project's two-part problem statement:

1. Automatically **generate** HamNoSys from visual input (image/video).
 2. Automatically **verify** that the generated notation is correct (via an avatar).
-

2. Big-Picture Architecture (Figure 1)



Everything downstream of "combine predictions" exists because there's no way to *know* the four modules got it right – the avatar is the project's stand-in for a labeled test set.

3. Phase 1 – Handshape Detection (the ML module)

HamNoSys handshape is **not one label** – it's a small combinatorial system. The report breaks it into a **tree of 5 nodes**:

Node	Meaning	Example values
A – Basic Handshape	overall hand shape family	fist, flat hand, two-finger, spread, pinch, curved (split into BASIC-1 / BASIC-2 sub-families)
C – Thumb Configuration	where the thumb sits	extended out / across palm / open
D – Finger-Type	how fingers are bent	straight, bent, hooked, double-bent
E – Finger States	which finger is doing the work	thumb, index, middle, ring/pinky, or a contact between two fingers
X – Extra	fine contact detail	fingertip, fingernail, finger-pad, finger-base contact

Prediction is progressive, not simultaneous: predict A first → based on A's result, *conditionally* decide whether to run C, then D, then E, then X. Not every path uses every node (Figure 2 shows legal paths like A-C-E, A-D-E, A-C-D-E, A-E, A-D, A-C). This guarantees the output can never be a structurally impossible combination.

Why hierarchical instead of one flat "predict everything at once" classifier?

- Fewer classes per model = each classifier has an easier, more separable job → less confusion between visually similar hand shapes.
- It mirrors how HamNoSys itself is actually built (compositional, not a flat list).
- It's directly enforceable for validity – a flat classifier could output any combination, including nonsense ones.

How it's trained: SVM / **XGBoost** (primary) and **Random Forest** classifiers, one per node, trained on features extracted from MediaPipe's 21 landmarks × 3 coordinates = **63 numerical features per hand image**.

Math here: essentially none beyond standard ML classification – the "math" in this module is entirely in how the *features* were engineered (joint angles, inter-finger distances) before being fed to the classifiers. The real formulas live in Orientation/Location/Movement below.

4. Phase 2 – Orientation Detection (pure geometry)

Orientation = which way the palm/fingers are pointing. **No ML** — this is computed straight from vector geometry.

4.1 Which "view" are we in?

HamNoSys defines orientation relative to a viewpoint: **Bird view, Signer view, Right view** (Palm model is the 4th descriptor family).

- **Bird view** if the wrist's vertical position is above eye level (wrist-y vs eye-y comparison).
- **Right view** if the wrist is to the right side of the right shoulder (wrist-x vs shoulder-x comparison).
- **Otherwise** → **Signer view** (the default, front-facing case).

4.2 The palm-direction vector

$\text{vec}(p) = L_middle_base - L_wrist$ (points from the wrist up through the hand)
 $\hat{n} = (-p_y, p_x)$ (rotate $\text{vec}(p)$ 90° , standard 2D perpendicular)

The angle this vector makes with the global coordinate axes tells you which way the palm/fingers point. Coordinates are normalized first, so the answer doesn't depend on how far the camera is or who's signing.

4.3 Mapping the angle to 1 of 8 directions

HamNoSys defines **8 finger directions**: right, up-right, up, up-left, left, down-left, down, down-right.

The math/"proof": a circle is 360° . Split evenly among 8 directions → $360/8 = 45^\circ$ per slice. This is the *only* partition where no direction gets an unfair advantage. Each slice is centered on its canonical angle ($0^\circ, 45^\circ, 90^\circ, \dots$) with a **$\pm 22.5^\circ$ tolerance**, giving boundaries at $\approx 22.5^\circ, 67.5^\circ, 112.5^\circ, 157.5^\circ$, etc. (The code rounds these to the nearest integer: 22, 67, 112...) A small filtering threshold ignores tiny angle wobbles caused by landmark noise, so the predicted direction doesn't flicker between two neighboring octants for a hand that's basically holding still.

5. Phase 3 – Location Detection (6 sub-modules, all geometry)

Location = *where on/around the body* the sign happens. This is the biggest, most detailed module — six sub-parts.

5.1 Head & Face Location

- 1. MediaPipe finds the face bounding box (hairline → chin).
- 2. Normalize the vertical position of any contact point:

$$\text{rel_y} = (\text{y} - \text{face_top}) / \text{face_height} \qquad \rightarrow \text{always in } [0, 1]$$

- 3. Slice that 0–1 range into anatomical zones, using **real facial-proportion ratios**, not arbitrary numbers (this is what Figures 3 & 4, the "rule of thirds" face diagrams, are illustrating):

rel_y range	HamNoSys label
< 0.10	hamheadtop
0.10–0.18	hamhead
0.18–0.28	forehead
0.28–0.36	eyebrow
0.36–0.44	hameyes
0.44–0.55	hamnose
0.55–0.65	hamnostrils
0.65–0.777	lips
0.777–0.888	teeth
0.888–1.00	chin
just below face box	hamunderchin
further below	hamneck

Notice 0.777 and 0.888 aren't random — the lower third of the face (lips/teeth/chin) is itself split into three roughly equal functional sub-units.

- 4. Horizontal position uses the same trick: $\text{rel_x} = (\text{x} - \text{face_left}) / \text{face_width}$, split into **5 equal bands**: outer 20% each side = ear/earlobe, next 20% each side = cheek, central 20% = midline face (nose/lips/chin).

5.2 Contact Types (how two points are touching)

All thresholds below are **fractions of normalized body height** (since MediaPipe normalizes to roughly 1.0 ≈ full body height), converted to real-world centimeters to justify the number:

Label	Rule	Real-world justification
-------	------	--------------------------

Label	Rule	Real-world justification
hamtouch	distance $d \leq \varepsilon = \mathbf{0.02}$	avg finger width $\approx 1.5\text{--}2\text{cm}$, avg height $\approx 170\text{cm} \rightarrow 2/170 \approx 0.012$; rounded up to tolerate finger curvature/noise
hamclose	$0.02 < d \leq \mathbf{0.05}$	$0.05 \times 170\text{cm} \approx 8.5\text{cm}$ — "near but not touching"
hambrushing	close range + motion $\Delta p = p_{t+1} - p_t > \mathbf{0.01}$	$\sim 1\text{--}2\text{cm}$ of real movement — enough to be intentional, not jitter
hambehind	depth difference $ z_1 - z_2 > \mathbf{0.03}$	torso depth $\approx 30\text{--}35\text{cm} \rightarrow 0.03 \approx 1\text{cm}$ real separation
hamcross	the left/right ordering of the two hands flips sign between frames, and depth confirms real overlap	depth check stops false positives from simple on-screen 2D overlap
haminterlock	several fingertip pairs stay close while differing in depth	distinguishes "laced together" from "flat touching"

The distance formula used everywhere: $d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

5.3 Finger Location Detection

- "Active finger" = whichever finger is most extended relative to the palm centroid: $d_i = \|p_{tip_i} - p_{palm}\|$.
- MediaPipe landmark index ranges: **thumb 1-4, index 5-8, middle 9-12, ring 13-16, pinky 17-20**.
- Once the active finger is known, *where on it* the contact happens is classified too: **hamfingertip** (nearest the tip), **hamfingermidjoint** (finger is bent, joint touches), **hambetween** (roughly equidistant from two adjacent fingers), **hamfingertipad** (palm-facing surface), **hamfingernail** (back/dorsal surface — found by comparing the "fingertip→palm" direction against the "fingertip→contact point" direction).

5.4 Hand Location Detection (where in the camera frame)

- Centroid of all 21 hand landmarks: $C = (C_x, C_y)$, the plain average of all the x's and all the y's.
- Horizontal thirds: $C_x < 0.33 \rightarrow$ **handleft**; $0.33\text{--}0.66 \rightarrow$ **handcenter**; $> 0.66 \rightarrow$ **handright**.
- Vertical thirds: $C_y < 0.33 \rightarrow$ **handup**; $> 0.66 \rightarrow$ **handdown**.
- **Majority voting** across frames: take the most common label (*mode*) over a short clip so one noisy frame can't flip the whole prediction — e.g., Figure 11 shows `{ 'hampalm': 3, 'hamthumbside': 1 }` $\rightarrow$ final answer = **hampalm** because it won 3-to-1.
- Everything here is scale-invariant because coordinates already live in $[0, 1]$ — irrelevant whether the camera is 1m or 3m away, 480p or 4K.

5.5 Upper Body Location Detection (torso/shoulder/elbow/wrist zones)

Uses MediaPipe **Pose** landmarks: shoulders (LS, RS), elbows (LE, RE), wrists, hips (LH, RH).

The core trick used throughout this whole sub-module: never threshold on raw distance — always divide by a body-scale reference first, so a tall person and a short person get identical classifications.

```
shoulder_width = d( LS, RS )
torso_height    = hip_y - shoulder_y
d_norm = d( A, B ) / scale          (scale = shoulder width, arm length, or torso height)
```

Region	Rule	Label
Shoulder	d_norm(C, shoulder) < 0.25, then split by Cy vs shoulder_y	hamshouldertop (above) / hamshoulders (at line)
Elbow	d_norm(C, elbow) < 0.20	hamelbowinside
Upper arm	hand is laterally outside the torso (	Cx-Tx
Lower arm	same lateral condition, Cy between elbow & wrist	hamlowerarm
Wrist	d_norm(C, wrist) < 0.10 **and** d_norm(C, elbow) > 0.40	hamwristback
Torso	rel = (Cy - shoulder_y) / torso_height	rel < 0.35 → hamchest · 0.35-0.70 → hamstomach · ≥ 0.70 → hambelowstomach

5.6 Arm & Space Location

θ = elbow angle (Shoulder-Elbow-Wrist), via the cosine rule:

$$\theta = \cos^{-1} [(S-E) \cdot (W-E) / (\|S-E\| \|W-E\|)]$$

R = extension ratio = $\|S-W\| / (\|S-E\| + \|E-W\|)$ → R → 1 for a fully straight arm

- **hamdoublebent**: both elbows $\theta < 120^\circ$ (natural elbow flex range is $\sim 30^\circ$ - 160° , so "bent" = lower half of that range).
- **hamarmextended**: $\theta > 160^\circ$ **and** $R > 0.95$ (angle *and* ratio both agree it's a straight line).
- **hamlrat** (hand at body midline): lateral offset $xoffset = |Wx-Tx| < 0.04$ (tight tolerance — shoulder width is normally ~ 0.35 - 0.40 , so 0.04 is a narrow "dead center" band).
- **hamlrbeside** (hand beside the body): $xoffset > 0.10$ **and** $zoffset < 0.08$ (far sideways, but not pushed forward — hanging at the side, not out signing).

- **hamneutralspace:** $z_{offset} > 0.05$ (hand has moved forward of the torso into the space in front of the body, where most actual signing happens).

6. Phase 4 – Movement Detection (2 sub-modules, all geometry)

6.1 Movement-1 – instantaneous direction / speed / repetition / size

The hand's path is just a list of 3D points over time: $p_t = (x_t, y_t, z_t)$.

$\Delta p_t = p_{\{t+1\}} - p_t$ (frame-to-frame displacement)

$(dx, dy, dz) = \text{mean of all } \Delta p_t \text{ over the clip (net "average direction" of travel)}$

$\varepsilon = 0.02$ (dead-zone: ignore anything smaller – camera jitter)

Direction rules (each axis checked against $\pm\varepsilon$):

Axis condition	Label
$dx > \varepsilon$	hammover(right)
$dx < -\varepsilon$	hammovei(left)
$dy < -\varepsilon$	hammoveu(up)
$dy > \varepsilon$	hammoved(down)
$dz < -\varepsilon$	hammovei(toward body)
$dz > \varepsilon$	hammoveo(away from body)

Multiple axes triggering together stack into compound labels (e.g., inward + left = **hammoveil**).

Speed: $\bar{v} = \text{mean}(\|\Delta p_t\|)$ – average per-frame displacement magnitude. $\bar{v} < 0.01 \rightarrow \text{**hamslow**}$; $\bar{v} > 0.03 \rightarrow \text{hamfast}$.

Repetition: track the sign of velocity along the dominant axis ($s_t = \text{sign}(v_t)$). Every time $s_t \cdot s_{\{t+1\}} < 0$, that's a direction reversal. 1 reversal $\rightarrow$ **hamrepeatreverse**; several $\rightarrow$ **hamrepeatcontinue**; many/frequent $\rightarrow$ **hamrepeatcontinueseveral**.

Size: $A = \max(p_t) - \min(p_t)$ – the total spatial range covered. $A < 0.02 \rightarrow \text{**hamsmallmod**}$; $A > 0.08 \rightarrow \text{hamlargemod}$; near-zero $\rightarrow$ **hamnomotion**.

6.2 Movement-2 – shape/quality of the whole trajectory

Movement-1 only looks frame-to-frame. Movement-2 looks at the **whole path's geometry**.

$$D = \|p_N - p_1\| \quad (\text{net displacement: straight line, start to end})$$
$$L = \sum \|p_{i+1} - p_i\| \quad (\text{total path length actually traveled})$$

- **Straight-line motion:** $D \approx L$ (you traveled exactly as far as you ended up).
- **Circular / looped motion:** $D \rightarrow 0$ while L stays large (you end up back where you started, but covered real distance). This single comparison is the core "proof" behind detecting circular motion.

Curvature (turning angle):

$$\theta_i = \cos^{-1} (v_i \cdot v_{i+1} / (\|v_i\| \|v_{i+1}\|)) \quad (\text{angle between consecutive direction vectors})$$

Average θ_i over the whole path $\rightarrow$ large mean = curvy/zigzag path, small mean = mostly straight.

Which plane the motion lives in (XY / XZ / YZ): compare how much the trajectory varies along each axis – whichever two axes show the most variance define the plane (e.g., mostly X & Y variance = a flat circle facing the camera; X & Z = a circle drawn in depth).

Clockwise vs. counter-clockwise: track each point's angular position around the trajectory's centroid, $\alpha_i = \tan^{-1}((y_i - \bar{y}) / (x_i - \bar{x}))$ (literally like a clock hand angle), then check whether that angle is increasing or decreasing over time. $d\alpha/dt < 0 \rightarrow$ clockwise.

"Stirring" (rotation without travel): $D \approx 0$, $L > 0$, and mean turning angle is large $\rightarrow$ rotated in place, like stirring a pot.

Oscillation: the sign of velocity along the dominant axis keeps flipping – same reversal-counting idea as Movement-1's repetition detector, but used here to flag a "wiggling" quality of motion.

7. Validation Layer

Every predicted piece (handshape + orientation + location + movement) is checked against a **reference CSV lexicon** that lists which combinations are actually legal HamNoSys (plus their hex codes). Anything that doesn't match a legal entry gets flagged/corrected. This is what stops the system from ever outputting "grammatically impossible" notation.

8. Avatar-Based Verification (how the project grades itself)

1. Validated HamNoSys string → auto-converted to **SiGML** (an XML sign-animation script format).
2. SiGML fed into the **JASigning** avatar player → renders a 3D avatar performing the predicted sign.
3. The avatar's landmark-derived feature vector is compared against the *original* input's feature vector using **cosine similarity**:

$$\cos_sim(A, B) = (A \cdot B) / (\|A\| \|B\|)$$

1.0 = identical direction/pattern of features, 0 = unrelated. The report's worked example gets **0.8812** cosine similarity, with **RMSE/RMSD = 0.2781** (Figure 6).

Why cosine similarity specifically? It compares the *angle/pattern* between two feature vectors rather than their raw magnitude — so it stays meaningful even though the avatar's proportions and the real human hand's proportions aren't identical in absolute scale.

This avatar loop is effectively the project's **only quantitative accuracy check** for the whole pipeline (beyond the CSV lexicon validity check) — it stands in for a proper labeled test set, which doesn't exist for this task.

9. Dataset & Platform

- **No public dataset** links raw sign images/video directly to HamNoSys symbols, so the team built one — **only for handshape**, since that's the piece that needed a trainable classifier.
 - **149** manually-defined, structurally valid handshape combinations.
 - 1 hand-collected seed image per combination → augmented (rotation $\pm 20^\circ$, scaling, horizontal flip, translation, brightness change, noise) into **21 variants each** → **3,000+ images total**.
 - A **CSV mapping table** ties every gesture combo to its official HamNoSys symbol + hex code — this is the ground truth used both to train the handshape models *and* as the lexicon for the validation layer in §7.
- **Orientation / Location / Movement have zero trained models** — a deliberate choice, because building a frame-by-frame annotated video dataset for those would be extremely labor-intensive and none exists publicly. Geometry needs no labeled examples.
- **Platform:** Google Colab Pro (GPU acceleration, long runtimes, high-memory VMs).
- **External verification tools:** HamNoSys Input Tool (reference/checking interface), JASigning (SiGML → avatar animation), LinguaSign (sample SiGML repository).
- **Software stack:** MediaPipe (landmarks) · OpenCV (image ops) · scikit-learn (preprocessing/metrics) · XGBoost + Random Forest (handshape classifiers) · pandas (CSV/data handling) · NumPy (vector math) · Matplotlib/Seaborn (plots) · Joblib (model save/load).

10. Results Recap (what each figure demonstrates)

Figure	What it shows
5	Handshape module correctly predicts hamceel2, hamfingerstraightmod, hamindexfinger, hammiddlefinger from a single input image
6	Avatar vs. original hand – cosine similarity 0.8812, RMSE/RMSD 0.2781
7	Orientation module output: ('bird', 'hamextfingero', 'hampalmd')
8	Head/face module correctly labels a chin-touch as hamteeth
9–10	Upper body module: hand-on-torso image → hambelowstomach , confidence 0.8
11	Hand location module with majority voting → hampalm
12	Finger location module → hampinky
13	Contact type module → hamtouch (distance ≈ 0.0)
14	Arm/space module → hamarmextended
15	Movement module full summary: direction, speed, force, halt, size, growth, repeat, path all predicted together
16	Final integrated output – all four modules' predictions shown together for one gesture

11. Threshold Cheat-Sheet (the exact numbers an examiner will probe)

Threshold	Value	Used for
ϵ (touch)	0.02	hamtouch cutoff
close range	0.02–0.05	hamclose
ϵ (movement dead-zone)	0.02	ignore noise in Movement-1
brushing motion	$\Delta p > 0.01$	hambrushing
behind (depth)	$ \Delta z > 0.03$	hambehind
slow speed	$\bar{v} < 0.01$	hamslow
fast speed	$\bar{v} > 0.03$	hamfast

Threshold	Value	Used for
small movement size	$A < 0.02$	hamsmallmod
large movement size	$A > 0.08$	hamlargemod
shoulder proximity	$d_norm < 0.25$	hamshoulders/top
elbow proximity	$d_norm < 0.20$	hamelbowinside
wrist proximity	$d_norm < 0.10$ (and elbow $d_norm > 0.40$)	hamwristback
chest/stomach/below split	$rel < 0.35$ / $0.35-0.70$ / ≥ 0.70	torso zones
double-bent elbow	$\theta < 120^\circ$	hamdoublebent
extended arm	$\theta > 160^\circ$, $R > 0.95$	hamarmextended
body midline	$xoffset < 0.04$	hamlrat
beside body	$xoffset > 0.10$, $zoffset < 0.08$	hamlrbeside
neutral signing space	$zoffset > 0.05$	hamneutralspace
hand-frame thirds	0.33 / 0.66	handleft/center/right, handup/down
face vertical zones	0.10 / 0.18 / 0.28 / 0.36 / 0.44 / 0.55 / 0.65 / 0.777 / 0.888	head→chin zones
face horizontal zones	0.20 / 0.40 / 0.60 / 0.80	ear/cheek/central
orientation octants	45° slices, $\pm 22.5^\circ$ tolerance	8 finger directions
handshape dataset	149 combos $\times$ 21 augmentations $\approx$ 3,000+ images	training data

12. Likely Viva Questions + Model Answers

Q1: Why is handshape done with ML while orientation/location/movement are rule-based?

No public dataset links sign video to HamNoSys labels. Building one manually for a single, controlled image-classification task (handshape) was feasible; annotating continuous video frame-by-frame for orientation/location/movement was not. Geometry from MediaPipe landmarks needs no training data at all, so it filled the gap.

Q2: Why hierarchical classification for handshape instead of one flat classifier?

Predicting five simpler sub-attributes (basic shape → thumb → finger-type → finger-state → extra) in sequence is more accurate than one giant multi-class model, reduces confusion between visually

similar handshapes, and – because the prediction path is conditional – the output can never be a structurally invalid combination.

Q3: Why normalize coordinates everywhere?

So every threshold works regardless of camera distance, image resolution, or the signer's body size. MediaPipe already outputs [0,1]-normalized coordinates; the project additionally divides distances by body-specific reference lengths (shoulder width, torso height, face height) for the body/face modules.

Q4: Why cosine similarity for avatar verification, not Euclidean distance?

Cosine similarity compares the *direction/pattern* of two feature vectors, not their raw magnitude – so it stays meaningful even when the avatar's hand proportions differ from the real signer's.

Q5: Why exactly 8 orientation directions at 45°?

360° split evenly among 8 directions is the only unbiased partition (no direction gets a bigger slice). This also matches HamNoSys's own defined set of 8 palm/finger directions.

Q6: What's the difference between Movement-1 and Movement-2?

Movement-1 = instantaneous, frame-to-frame stats: direction, speed, repetition, size, computed purely from Δp_t . Movement-2 = the shape of the *entire* trajectory over time – curvature, circularity, rotation direction, oscillation – computed by comparing net displacement to total path length and by measuring turning angles.

Q7: Why is a dead-zone threshold needed in movement detection?

Landmark tracking is never perfectly still frame-to-frame – small camera/model noise would otherwise be misread as constant "movement." $\epsilon = 0.02$ filters that out.

Q8: How does the system decide its own predictions were wrong?

The avatar reconstruction step (Figure 1's decision diamond) – if the avatar-vs-original cosine similarity is too low, the pipeline loops back to "Refine Models."

Q9: What serves as ground truth, since no public HamNoSys-labeled dataset exists?

A self-built CSV mapping table linking gesture combinations to valid HamNoSys symbol strings and hex codes – used both to train the handshape classifiers and as the lexicon for the validation layer.

Q10: Why use both Random Forest and XGBoost?

XGBoost is the primary classifier for the hierarchical handshape nodes and also acts as a fallback/global classifier to resolve ambiguous combinations; Random Forest was used and compared for balance across categories during model selection.

Q11: What's the real-world meaning of $\epsilon = 0.02$ for "touch"?

Average finger width (1.5–2cm) divided by average human height (170cm) ≈ 0.012 ; rounded up slightly to 0.02 to tolerate finger curvature and landmark tracking noise.

Q12: Why does Location split into six separate sub-modules instead of one location model?

Because HamNoSys location is itself compositional – a sign can be located on the face, on/near the

other hand, on the body, or in free signing space – and each of those needs different anatomical reference points and different normalization scales (face height vs. shoulder width vs. torso height).

13. Honest Limitations (know these – examiners like to hear you can self-critique)

- Only handshape has a trained, dataset-backed ML model; orientation/location/movement rely entirely on hand-picked geometric thresholds that were **not** validated against a large labeled test set – they could misbehave on unusual body proportions or camera angles.
 - No aggregate, dataset-wide accuracy numbers are reported for orientation/location/movement – evidence is qualitative (per-figure examples) plus one avatar cosine-similarity example (0.88).
 - The handshape dataset is one real seed image per class, heavily augmented synthetically – not sourced from many different real signers, so real-world generalization is untested.
 - Continuous/connected signing (transitions, co-articulation, two-hand overlap) is flagged in the introduction as the field's hardest unsolved problem, and the shown results are still largely per-frame/per-gesture snapshots rather than a full continuous-sentence demo end-to-end.
-

14. Quick Glossary

- **HamNoSys** – Hamburg Notation System, symbolic writing system for signs.
- **SiGML** – Signing Gesture Markup Language, an XML format used to drive avatar animation.
- **MediaPipe** – Google's computer-vision library used here for hand (21 landmarks), face, and body-pose landmark detection.
- **Landmark** – a tracked (x, y, z) keypoint on the hand/face/body.
- **Normalized coordinate** – a coordinate rescaled into $[0, 1]$ relative to some reference (frame size, face box, body scale) so it's independent of camera distance/resolution.
- **Cosine similarity** – a measure of how similar the *direction* of two vectors is, regardless of their magnitude; 1 = identical, 0 = unrelated.
- **Dead-zone / epsilon (ϵ)** – a small threshold below which a measured change is treated as noise, not a real signal.